

DEDAN KIMATHI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF COMPUTER SCIENCE AND INFORMATICS
DEPARTMENT OF COMPUTER SCIENCE
CCS 3101: INTRODUCTION TO ARTIFICIAL INTELLIGENCE
AI CAPSTONE PROJECT REPORT
INTELLIGENT HEALTHCARE DIAGNOSTIC ASSISTANT

GROUP MEMBERS

No.	Name	Role
1	Loren Deklerk	Project Manager
2	Andrew Nyawira	Frontend Developer
3	Kepha Mirera	Backend Developer
4	Stephen Ongoma	DevOps & QA Specialist
5	Lewis Ngacha	AI/ML Specialist

Course: CCS 3101 – Introduction to Artificial Intelligence

Project: Intelligent Healthcare Diagnostic Assistant

Academic Year: 2026

DECLARATION

We declare that this project report presents the work carried out by the above-named group as part of the requirements for the CCS 3101: Introduction to Artificial Intelligence course at Dedan Kimathi University of Science and Technology.

The project was developed as an end-to-end intelligent healthcare diagnostic system incorporating the Artificial Intelligence concepts covered throughout the course. The implementation followed the requirements and guidance provided in the AI Capstone Project Lab Manual.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our lecturer for providing the guidance, technical requirements, and learning framework that enabled us to undertake this Artificial Intelligence capstone project.

We also appreciate the contributions of all members of the project team. Each member contributed to the planning, development, integration, testing, quality assurance, and completion of the system.

Finally, we acknowledge the value of the concepts covered throughout the Artificial Intelligence course, which provided the theoretical foundation for the development of the Intelligent Healthcare Diagnostic Assistant.

ABSTRACT

The Intelligent Healthcare Diagnostic Assistant is an end-to-end Artificial Intelligence system designed to assist with healthcare diagnosis, severity assessment, and treatment planning based on patient symptoms and vital signs.

The system integrates multiple Artificial Intelligence techniques into a unified platform. These include intelligent agents, knowledge-based reasoning, First-Order Logic and inference, Bayesian probabilistic reasoning, supervised machine learning, deep neural networks, fuzzy logic, and AI planning.

The intelligent agent acts as the central coordinator of the system. It receives patient information, communicates with the different diagnostic modules, collects their results, evaluates the information, determines the urgency of the patient's condition, and produces a structured diagnostic report.

The knowledge-based module applies logical rules through forward and backward chaining. The Bayesian diagnostic module reasons under uncertainty using probability theory and Naïve Bayes. The machine learning module compares Decision Tree, Random Forest, and Gradient Boosting classifiers. The neural network module uses a Multi-Layer Perceptron implemented using TensorFlow/Keras. The fuzzy logic module assesses the severity of a patient's condition using temperature, heart rate, and symptom count. Finally, the treatment planner uses STRIPS-style planning and Breadth-First Search to generate an appropriate sequence of treatment actions.

The project demonstrates how different Artificial Intelligence techniques can be integrated into a single practical application rather than relying on a single AI approach. The completed system provides diagnostic recommendations, confidence information, severity assessment, and treatment planning while demonstrating the practical application of concepts studied throughout the course.

1. INTRODUCTION

Artificial Intelligence has increasingly become an important technology in healthcare because of its ability to process large amounts of information, identify patterns, reason under uncertainty, and support decision-making.

The purpose of this capstone project was to develop an intelligent healthcare diagnostic system that combines several Artificial Intelligence techniques into one integrated application.

The project, titled **Intelligent Healthcare Diagnostic Assistant**, was designed according to the AI Capstone Project Lab requirements. The project roadmap covered topic selection and proposal development, data collection and preprocessing, core AI development, integration and application development, evaluation and optimization, and finally system demonstration and reporting.

The system does not rely on a single AI technique. Instead, different AI modules act as specialist components. Each module provides its own analysis and confidence information, after which the intelligent agent combines the outputs to produce a final recommendation. This follows the capstone's central concept that no individual module is responsible for the complete diagnostic decision.

2. PROBLEM STATEMENT

Healthcare diagnosis often requires the analysis of several types of information, including symptoms, vital signs, patient history, and previous medical knowledge.

Traditional rule-based systems may struggle when information is incomplete or uncertain.

Similarly, purely statistical or machine learning approaches may provide predictions without explicitly representing medical knowledge or reasoning about treatment actions.

There was therefore a need to develop a system capable of combining different Artificial Intelligence approaches to support healthcare diagnosis.

The project addresses this problem by developing an integrated healthcare diagnostic assistant capable of:

- Receiving patient symptoms and vital signs.
- Applying logical medical rules.
- Reasoning probabilistically under uncertainty.
- Using machine learning to recognize patterns in patient data.
- Applying deep learning to diagnostic classification.
- Assessing the severity of a patient's condition using fuzzy logic.
- Generating a sequence of recommended treatment actions using AI planning.
- Combining the outputs of different AI modules through an intelligent agent.

3. PROJECT OBJECTIVES

3.1 General Objective

To develop an integrated Artificial Intelligence healthcare diagnostic assistant that combines multiple AI techniques to provide diagnostic recommendations, severity assessment, and treatment planning.

3.2 Specific Objectives

The project aimed to:

1. Develop an intelligent agent capable of coordinating the different AI modules.
2. Implement a medical knowledge base using logical rules and inference.
3. Implement Bayesian probabilistic reasoning for diagnosis under uncertainty.
4. Develop machine learning classifiers for disease prediction.
5. Develop a deep neural network for diagnostic classification.
6. Implement fuzzy logic for healthcare severity assessment.
7. Develop an AI treatment planner using state-space planning.
8. Integrate all modules into one functioning healthcare diagnostic system.
9. Test and evaluate the performance of the AI components.
10. Demonstrate the practical application of Artificial Intelligence concepts covered in the course.

4. SCOPE OF THE PROJECT

The project focuses on developing a prototype intelligent healthcare diagnostic assistant.

The system accepts patient information such as:

- Patient identification.
- Symptoms.
- Age.
- Temperature.
- Heart rate.
- Blood pressure.

The system then processes the information through multiple AI modules.

The scope includes:

- Intelligent agent architecture.
- Knowledge representation.
- Forward chaining.
- Backward chaining.
- Bayesian reasoning.
- Supervised machine learning.
- Deep learning.

- Fuzzy logic.
- AI planning.
- System integration.
- Testing and evaluation.

The system is intended as an **AI-based decision-support prototype** and not as a replacement for qualified medical professionals.

5. SYSTEM REQUIREMENTS

5.1 Software Requirements

The project development environment included:

- Python 3.9 or later.
- Git.
- VS Code and/or Jupyter.
- Python pip package manager.
- Required Python libraries and project dependencies.
- Machine learning and deep learning frameworks required by the implementation.

The project manual specifies Python 3.9+, Git, VS Code or Jupyter, and installation of project dependencies through `requirements.txt`.

5.2 Development Environment

The system was developed using a modular architecture in which the different Artificial Intelligence components were implemented separately and then integrated.

The recommended project workflow involved:

1. Obtaining the project repository.
2. Creating a Python virtual environment.
3. Installing dependencies.
4. Developing individual modules.
5. Testing modules independently.
6. Integrating the modules.
7. Running the complete application.
8. Evaluating the resulting system.

The project manual specifically recommends testing individual modules independently before integrating them into the main application.

6. SYSTEM ARCHITECTURE

The system follows a modular Artificial Intelligence architecture.

At the center of the system is the **Intelligent Agent**, which coordinates the other AI components.

The major components are:

1. Intelligent Agent Core.
2. Medical Knowledge Base and Inference Engine.
3. Bayesian Network and Probabilistic Reasoning.
4. Machine Learning Diagnostic Classifier.
5. Deep Neural Network Diagnostic Model.
6. Fuzzy Logic Severity Assessor.
7. AI Treatment Planner.
8. Evaluation and visualization components.
9. Application/interface layer.

6.1 High-Level Architecture

The system follows the general flow:

Patient Data → Intelligent Agent → AI Modules → Results → Agent Reasoning → Severity Assessment → Treatment Planning → Final Report

The intelligent agent follows a:

Perceive → Think → Act

architecture.

Patient information is first perceived by the agent. The agent then invokes the registered AI modules and collects their diagnostic results. Finally, it combines the information and produces a structured output.

7. TEAM ORGANIZATION AND RESPONSIBILITIES

7.1 Loren Deklerk – Project Manager

The Project Manager was responsible for:

- Coordinating the overall project.
- Organizing project activities and deadlines.
- Monitoring progress across team members.
- Coordinating integration between different project components.
- Maintaining project documentation.
- Ensuring that the implementation followed the lecturer's requirements.
- Coordinating testing and final preparation.
- Ensuring completion of the final report and presentation.

7.2 Andrew Nyawira – Frontend Developer

The Frontend Developer was responsible for:

- Developing the user-facing application layer.

- Designing the interaction between users and the diagnostic system.
- Providing mechanisms for entering patient information.
- Presenting diagnostic results and recommendations.
- Supporting the integration of the interface with the backend services.

7.3 Kepha Mirera – Backend Developer

The Backend Developer was responsible for:

- Developing the application backend.
- Managing communication between the application layer and AI components.
- Supporting patient-data processing.
- Integrating backend functionality with the diagnostic modules.
- Supporting the overall system integration.

7.4 Stephen Ongoma – DevOps & QA Specialist

The DevOps and QA Specialist was responsible for:

- Supporting the development environment.
- Managing version control and project workflow.
- Supporting deployment and system execution.
- Conducting quality assurance.
- Testing individual modules.
- Testing integrated system functionality.
- Identifying and documenting errors.
- Supporting reliability and consistency of the final system.

7.5 Lewis Ngacha – AI/ML Specialist

The AI/ML Specialist was responsible for:

- Implementing and integrating the Artificial Intelligence components.
- Developing machine learning models.
- Supporting the neural network implementation.
- Implementing AI reasoning techniques.
- Supporting Bayesian reasoning.
- Supporting fuzzy logic and AI planning.
- Evaluating AI model performance.
- Supporting the integration of AI outputs into the intelligent agent.

8. SYSTEM IMPLEMENTATION

8.1 Intelligent Agent Core

The Intelligent Agent acts as the central coordinator of the healthcare diagnostic system.

The agent receives patient information through a patient percept structure containing information such as patient ID, symptoms, age, temperature, heart rate, and blood pressure. The agent implements three main processes:

Perceive

The `perceive()` process:

- Receives patient information.
- Stores the patient data in the agent's memory.
- Records the relevant timestamp.
- Updates the internal state.

Think

The `think()` process invokes each registered AI module and collects its analysis.

The agent therefore acts as a coordinator rather than relying on one diagnostic technique.

Act

The `act()` process:

- Combines diagnostic information.
- Considers confidence scores.
- Determines urgency.
- Generates recommendations.
- Produces the final structured report.

The lecturer's manual describes this as a model-based, goal-based agent and identifies the PEAS components as performance, environment, actuators, and sensors.

9. MEDICAL KNOWLEDGE BASE AND INFERENCE ENGINE

The Medical Knowledge Base acts as the rule-based reasoning component of the system.

It represents medical knowledge using:

- Facts.
- Rules.
- Conclusions.
- Certainty factors.

For example, a rule can represent a relationship between symptoms and a suspected disease.

A simplified representation is:

Fever AND Cough AND Loss of Smell AND Fatigue → COVID-19 Suspected

with an associated certainty factor.

The implementation applies two inference strategies.

9.1 Forward Chaining

Forward chaining is a data-driven reasoning technique.

The system begins with known patient symptoms and repeatedly applies rules whose conditions are satisfied until no additional conclusions can be generated.

The general process is:

Symptoms → Rules → New Facts → Diagnosis

9.2 Backward Chaining

Backward chaining is goal-driven.

The system begins with a suspected diagnosis and works backwards to determine whether the available symptoms satisfy the conditions necessary to support that diagnosis.

The process is:

Diagnosis → Required Conditions → Patient Symptoms

The lecturer's manual specifically requires the knowledge base to support both forward and backward chaining and certainty factors.

10. BAYESIAN NETWORK AND PROBABILISTIC REASONING

The Bayesian diagnostic module provides probabilistic reasoning.

Unlike the rule-based knowledge base, which uses explicit rules, the Bayesian module calculates the probability of possible diseases given observed symptoms.

The project applies the Naïve Bayes formulation:

$P(\text{Disease} \mid \text{Symptoms}) \propto P(\text{Disease}) \times P(\text{Symptom}_1 \mid \text{Disease}) \times P(\text{Symptom}_2 \mid \text{Disease}) \times \dots$

The main components include:

Prior Probability

The prior represents how common a disease is before considering the current patient's symptoms.

Likelihood

The likelihood represents how probable a particular symptom is given a disease.

Posterior Probability

The posterior represents the updated probability of a disease after considering the patient's symptoms.

The implementation uses logarithmic probabilities when multiplying multiple small probabilities to reduce numerical underflow.

The module returns the highest-probability diagnosis together with ranked diagnostic probabilities.

11. MACHINE LEARNING DIAGNOSTIC CLASSIFIER

The Machine Learning Diagnostic Classifier provides a data-driven approach to disease prediction.

The system uses a synthetic patient dataset because a real hospital database was not available for the project.

Each patient is represented using an **18-dimensional binary feature vector**, where:

- 1 represents a symptom being present.
- 0 represents a symptom being absent.

The module evaluates three machine learning algorithms:

1. Decision Tree.
2. Random Forest.
3. Gradient Boosting.

11.1 Decision Tree

The Decision Tree classifies patients by creating decision rules based on the most informative features.

Its major advantage is interpretability.

11.2 Random Forest

Random Forest combines multiple decision trees and uses their collective predictions.

This improves robustness and reduces reliance on a single tree.

11.3 Gradient Boosting

Gradient Boosting builds models sequentially, with each subsequent model attempting to correct errors made by previous models.

11.4 Model Selection

The three models are evaluated using cross-validation.

The best-performing model is selected automatically for final prediction.

The capstone manual specifies five-fold cross-validation as part of the evaluation approach.

12. DEEP NEURAL NETWORK DIAGNOSTIC MODEL

The Deep Neural Network module applies deep learning to disease classification.

The model uses a Multi-Layer Perceptron implemented using TensorFlow/Keras.

12.1 Network Architecture

The architecture consists of:

- Input layer: 18 neurons.
- Hidden Layer 1: 128 neurons.
- Batch Normalization.
- Dropout of 30%.
- Hidden Layer 2: 64 neurons.
- Batch Normalization.
- Dropout of 20%.

- Hidden Layer 3: 32 neurons.
- Batch Normalization.
- Output layer: 8 neurons with Softmax activation.

The architecture follows the structure specified in the capstone manual.

12.2 Techniques Used

ReLU Activation

ReLU introduces non-linearity into the neural network.

Softmax

Softmax converts the output into probabilities across the disease classes.

Dropout

Dropout reduces overfitting by randomly disabling neurons during training.

Batch Normalization

Batch normalization improves training stability.

Early Stopping

Early stopping prevents unnecessary training once validation performance stops improving.

L2 Regularization

L2 regularization helps control large model weights and reduce overfitting.

The neural network therefore provides another independent diagnostic perspective alongside the rule-based, probabilistic, and traditional machine learning modules.

13. FUZZY LOGIC SEVERITY ASSESSOR

The Fuzzy Logic Severity Assessor does not primarily determine the disease. Instead, it determines **how serious the patient's current condition is**.

The module uses three inputs:

1. Temperature.
2. Heart rate.
3. Number of symptoms.

It produces a severity score between **0 and 100**.

13.1 Fuzzification

The crisp patient measurements are converted into degrees of membership.

For temperature, the fuzzy categories include:

- Normal.
- Mild.
- High.
- Critical.

A patient measurement can belong partially to more than one category.

13.2 Rule Evaluation

Fuzzy IF-THEN rules are applied to the membership values.

For example:

IF temperature IS high AND heart rate IS elevated THEN severity IS high.

The strength of the rule is determined using fuzzy operations such as $\min()$ for AND conditions and $\max()$ for OR conditions.

13.3 Defuzzification

The fuzzy results are converted into a single numerical severity score using the centroid method.

The severity categories are:

Score	Severity
0–20	LOW
20–40	MILD
40–60	MODERATE
60–80	HIGH
80–100	CRITICAL

This approach allows the system to represent the uncertainty and gradual transitions that exist between different levels of illness severity.

14. AI TREATMENT PLANNER

The Treatment Planner is responsible for generating a sequence of actions based on the diagnosis and severity produced by the system.

The module applies **STRIPS planning** and **Breadth-First Search (BFS)**.

Each medical action is represented using:

- Preconditions.
- Delete effects.
- Add effects.
- Cost.
- Duration.

For example, an action may require a patient to have a particular suspected diagnosis before a specific medical action can be performed.

14.1 State-Space Planning

The planner starts from an initial patient state and searches for a sequence of actions capable of reaching the desired goal state.

The goal state can include:

- Treatment started.
- Vital signs monitored.

- Follow-up scheduled.

14.2 Breadth-First Search

BFS explores possible action sequences systematically until a valid path to the goal state is found.

A visited-state mechanism is used to prevent the planner from repeatedly exploring the same state.

The treatment planner therefore transforms the diagnostic output into a structured sequence of recommended actions.

15. SYSTEM WORKFLOW

The complete system follows the following workflow:

Step 1: Patient Input

The user provides patient information, including symptoms and vital signs.

Step 2: Agent Perception

The intelligent agent receives and stores the patient information.

Step 3: Parallel AI Analysis

The patient information is passed to the registered AI modules.

Step 4: Knowledge-Based Reasoning

The knowledge base applies logical rules using forward and backward chaining.

Step 5: Bayesian Reasoning

The Bayesian module calculates disease probabilities based on the observed symptoms.

Step 6: Machine Learning Prediction

The machine learning classifier predicts the most likely disease using learned patterns.

Step 7: Neural Network Prediction

The neural network independently evaluates the patient feature vector.

Step 8: Severity Assessment

The fuzzy logic module evaluates the patient's temperature, heart rate, and symptom count.

Step 9: Agent Decision

The intelligent agent combines the diagnostic information and determines the overall recommendation and urgency.

Step 10: Treatment Planning

The treatment planner generates a sequence of actions based on the diagnosis and severity.

Step 11: Final Report

The system presents a structured output containing diagnostic information, confidence, severity, recommendations, and treatment planning information.

16. TESTING AND EVALUATION

Testing was performed at both module and system levels.

16.1 Unit Testing

Each AI module was tested independently before integration.

The testing approach included:

- Providing known input cases.
- Running the module.
- Checking whether the output matched the expected behavior.
- Identifying errors.
- Correcting implementation problems.
- Re-running the test.

This approach follows the lecturer's recommendation that each module should be tested independently before integration into the main application.

16.2 Integration Testing

After individual modules were functional, they were connected through the intelligent agent.

Integration testing verified that:

- Patient information was correctly passed between components.
- AI modules received the required input.
- Module outputs were returned correctly.
- The agent successfully combined module results.

17. EVALUATION METRICS

The project evaluation follows the metrics specified in the capstone manual.

17.1 Accuracy

Accuracy measures the proportion of predictions that are correct.

Accuracy = Correct Predictions / Total Predictions

17.2 Precision

Precision measures how many positive predictions were actually correct.

Precision = TP / (TP + FP)

17.3 Recall

Recall measures how many actual positive cases were correctly identified.

Recall = TP / (TP + FN)

17.4 F1-Score

F1-score provides a balance between precision and recall.

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

17.5 Confusion Matrix

A confusion matrix compares predicted classes against actual classes and helps identify which diseases are commonly confused by the model.

17.6 ROC-AUC

ROC-AUC measures the ability of a classifier to distinguish between classes.

The lecturer's capstone manual identifies accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC as the required evaluation metrics.

18. EVALUATION RESULTS

The completed implementation was evaluated using the required machine learning and diagnostic testing procedures.

System Testing

Test Case	Input Condition	Expected Function	Result
1	Patient symptoms and vitals	Generate diagnosis	Passed
2	Fever and respiratory symptoms	Apply knowledge-based reasoning	Passed
3	Multiple symptoms	Generate probabilistic diagnosis	Passed
4	Patient feature vector	Generate ML prediction	Passed
5	Severe symptoms and abnormal vitals	Generate severity assessment	Passed
6	Diagnosis and severity	Generate treatment plan	Passed
7	Complete patient workflow	Generate final integrated report	Passed

The exact numerical evaluation values should be populated from the final outputs generated by the completed implementation.

19. RESULTS AND DISCUSSION

The completed project demonstrates the integration of multiple Artificial Intelligence techniques within one healthcare application.

One of the main strengths of the system is that different AI approaches provide complementary forms of reasoning.

The knowledge-based system provides explainable rule-based reasoning. Bayesian reasoning provides probabilistic interpretation of symptoms. Traditional machine learning provides data-driven classification, while the neural network provides a deep learning perspective.

The fuzzy logic component addresses a different aspect of the healthcare problem by estimating severity rather than simply predicting a disease.

The planning component then transforms diagnostic information into a sequence of actions.

The intelligent agent acts as the coordinator between these components, ensuring that the individual AI techniques contribute to one overall system.

This architecture demonstrates the practical advantage of combining different AI techniques.

Instead of expecting one algorithm to solve every part of the problem, each component specializes in a particular task.

20. CHALLENGES ENCOUNTERED

During the development and integration of the system, several technical challenges can arise when implementing a multi-module Artificial Intelligence application.

These include:

20.1 Module Integration

Integrating independently developed modules requires consistent interfaces and data formats.

20.2 Symptom Representation

Symptoms must be represented consistently so that the rule-based, Bayesian, machine learning, and neural network components interpret the same patient information correctly.

20.3 Model Training

Machine learning and neural network components require training data and appropriate preprocessing.

20.4 Overfitting

Machine learning and deep learning models may memorize training data instead of learning general patterns.

20.5 Probabilistic Computation

Multiplying many small probabilities can lead to numerical underflow, which is why logarithmic probability calculations are useful.

20.6 Fuzzy Logic

Membership functions and rules must be designed carefully to ensure that different patient conditions produce meaningful severity scores.

20.7 Planning Search

The treatment planner must avoid repeatedly exploring the same states. A visited-state mechanism is therefore required.

21. SOLUTIONS AND IMPROVEMENTS

Several design strategies were used to address the challenges associated with the project.

Modular Development

The system was divided into separate modules so that individual AI techniques could be developed and tested independently.

Standardized Data Representation

Patient symptoms were normalized into consistent representations before being processed by different AI components.

Cross-Validation

Five-fold cross-validation was used for machine learning model evaluation as specified by the project requirements.

Regularization

The neural network uses dropout, batch normalization, early stopping, and L2 regularization to improve generalization.

Numerical Stability

The Bayesian component uses logarithmic probabilities to reduce numerical problems associated with multiplying very small probabilities.

Visited States

The treatment planner uses state tracking to prevent repeated exploration of identical states.

Independent Testing

Individual modules were tested before being integrated into the complete application.

22. ETHICAL AND SAFETY CONSIDERATIONS

Because the project operates in the healthcare domain, ethical and safety considerations are important.

The system is designed as an **educational AI decision-support prototype** and should not be treated as a replacement for qualified healthcare professionals.

The system's recommendations should therefore be interpreted as AI-generated assistance rather than definitive medical diagnosis or treatment.

Additional considerations include:

- Protecting patient information.

- Avoiding unnecessary exposure of personal medical data.
- Clearly communicating uncertainty.
- Avoiding over-reliance on automated recommendations.
- Ensuring that healthcare professionals remain responsible for final medical decisions.
- Testing AI models before deployment in real healthcare environments.

The system should only be considered for real-world clinical use after extensive validation using appropriate clinical data, professional oversight, safety testing, and regulatory approval.

23. PROJECT ACHIEVEMENTS

The project successfully brought together several major Artificial Intelligence concepts into one integrated application.

The completed system incorporates:

- Intelligent agent architecture.
- PEAS framework.
- Knowledge representation.
- First-Order Logic.
- Forward chaining.
- Backward chaining.
- Certainty factors.
- Bayesian reasoning.
- Naïve Bayes.
- Supervised machine learning.
- Decision Trees.
- Random Forest.
- Gradient Boosting.
- Deep neural networks.
- Fuzzy logic.
- Fuzzy control.
- STRIPS planning.
- Breadth-First Search.
- Model evaluation.
- System integration.

The capstone manual identifies the completion and integration of all seven modules, evaluation of the machine learning components, generation of confusion matrices and comparison visualizations, end-to-end testing, documentation, and final presentation as the major final deliverables.

24. CONCLUSION

The Intelligent Healthcare Diagnostic Assistant demonstrates how multiple Artificial Intelligence techniques can be integrated to solve a complex real-world problem.

The system combines symbolic reasoning, probabilistic reasoning, machine learning, deep learning, fuzzy logic, and AI planning within a single intelligent-agent architecture.

The intelligent agent coordinates the individual modules and combines their outputs to generate a comprehensive recommendation. The knowledge base provides rule-based reasoning, the Bayesian module handles uncertainty, the machine learning and neural network modules identify learned patterns, the fuzzy logic component assesses severity, and the treatment planner generates a sequence of actions.

The project therefore demonstrates the practical application of the Artificial Intelligence concepts covered throughout CCS 3101.

Beyond demonstrating individual algorithms, the project highlights an important principle in Artificial Intelligence: different reasoning approaches can complement one another when integrated into a unified intelligent system.

25. RECOMMENDATIONS

The following improvements are recommended for future versions of the system:

1. **Use real, professionally validated healthcare datasets** instead of relying primarily on synthetic data.
2. **Increase the size and diversity of the training dataset** to improve model generalization.
3. **Include additional patient features**, such as medical history, age groups, medication history, and laboratory results.
4. **Improve explainability** by providing reasons behind AI-generated predictions.
5. **Expand the knowledge base** with additional medically validated rules.
6. **Improve the treatment planner** by incorporating more medical scenarios and constraints.
7. **Implement stronger privacy and security mechanisms** for patient information.
8. **Perform extensive validation with healthcare professionals** before considering real-world deployment.
9. **Introduce continuous model evaluation** to monitor changes in diagnostic performance.
10. **Develop a more comprehensive user interface** to improve usability and accessibility.

26. REFERENCES

1. Dedan Kimathi University of Science and Technology. **AI Capstone Project Lab Manual – CCS 3101 Introduction to Artificial Intelligence.**
2. Course materials for **CCS 3101: Introduction to Artificial Intelligence**, Dedan Kimathi University of Science and Technology.
3. Project source code and implementation developed by the project team.